

Instituto Federal de São Paulo - *campus* Barretos

Tecnologia em Análise e Desenvolvimento de

Sistemas

Desenvolvimento *Web* III (DW3A6)

Avaliação 1 - Parte 2

Introdução

Nesta avaliação, você deverá desenvolver um sistema simples sobre tarefas a realizar, de acordo com as especificações e restrições listadas neste documento.

Forma de Avaliação

A avaliação levará em consideração as seguintes características do sistema desenvolvido:

- Correto atendimento do sistema em relação às especificações fornecidas neste documento;
- Utilização correta dos conceitos trabalhados na disciplina;
- Funcionamento correto, eficiente e seguro do sistema desenvolvido;

Desenvolvimento

Esta avaliação deverá ser feita, de forma individual, apenas no laboratório onde a avaliação será aplicada.

Data de Entrega

O prazo máximo para a entrega do código-fonte do sistema desenvolvido é o dia 11/nov/2024 às 22:15.

Forma de Entrega

A entrega deverá ser realizada pelo aplicativo disponibilizado nos computadores do laboratório do IFSP.

1 Descrição geral do sistema

O sistema a ser desenvolvido nesta avaliação deve ser um sistema de usuário único, em que são exibidas tarefas a serem realizadas, cadastradas pelo próprio usuário. O sistema é composto de três partes:

- um *site* para interação com o usuário;
- uma API, à qual o *site* realiza requisições para obter/armazenar dados;
- um banco de dados, no qual os dados são armazenados e devem ser acessados apenas pela API.

Para mais detalhes, consulte os requisitos não funcionais.

2 Requisitos Funcionais

Os requisitos funcionais descrevem o comportamento do sistema e estão listados aqui em ordem aleatória. Todo o comportamento descrito aqui é obrigatório e deve ser realizado. Incrementos podem ser feitos, a critério do discente, desde que não alterem o comportamento dos requisitos.

RF01 - Tela inicial

O sistema deve exibir, na sua tela inicial, acessada pela rota padrão / do *site*, todas as atividades a serem realizadas, ordenadas das mais prioritárias para as menos prioritárias e, em seguida, em ordem alfabética. Para cada atividade exibida, é possível fazer sua edição (ver RF04), remoção (ver RF05) e marcar como realizada (ver RF03). Ainda nesta tela é possível criar uma nova atividade (ver RF02) e visualizar as atividades já realizadas (ver RF06).

Para listar os dados obtidos do banco de dados, o *site* deve acessar a rota / da API com uma requisição do tipo GET, de acordo com as regras de uma API Restful.

RF02 - Criação de atividade

Para criar uma atividade deve-se acessar a rota /**add** do *site*. Os dados obrigatórios para uma atividade são o seu título (texto único) e sua prioridade, que pode ser:

- Muito alta
- Alta
- Padrão
- Baixa
- Muito baixa

Opcionalmente, pode-se informar detalhes da atividade (texto).

Esta tela deve ser exibida com todos os controles de entrada em branco, e apresentar dois botões ao usuário: **Cancelar**, em que retorna-se à tela principal do sistema, e **Salvar**, em que, após validação dos dados, armazena-se eles na base de dados. Em caso de sucesso, voltar à tela inicial, avisar o usuário sobre o sucesso do armazenamento, e atualizar a lista de atividades. Em caso de falha, avisar o usuário sobre esta situação mas permanecer na página atual, sem limpar os dados digitados.

Para armazenar os dados no banco de dados, o *site* deve acessar a rota / da API com uma requisição do tipo POST, informando os dados no corpo da requisição realizada, de acordo com as regras de uma API Restful.

RF03 - Marcar atividade como realizada

Todas as atividades não realizadas, exibidas na tela inicial, devem possuir uma opção para marcar ela como realizada. Quando esta opção é ativada, essa informação deve ser armazenada no banco de dados, uma mensagem de sucesso/erro deve ser exibida ao usuário, e em caso de sucesso, a atividade não deve ser mais exibida como uma atividade a ser realizada na página inicial.

Para atualizar dados no banco de dados, o *site* deve acessar a rota / da API com uma requisição do tipo PUT, informando os dados no corpo e na rota da requisição realizada, de acordo com as regras de uma API Restful.

RF04 - Edição de atividade

Todas as atividades não realizadas, exibidas na tela inicial, devem possuir uma opção para editar seus dados. Quando esta opção é escolhida, deve-se acessar uma nova página na rota `/editar`, informando a tarefa a ser editada. Ao carregar esta página, os dados da tarefa devem ser preenchidos de acordo com o armazenado no banco de dados. Devem ser apresentados dois botões ao usuário: **Cancelar**, em que retorna-se à tela principal do sistema, e **Salvar**, em que, após validação dos dados, atualiza-se a tarefa informada com os dados informados na base de dados. Em caso de sucesso, voltar à tela inicial, avisar ao usuário sobre o sucesso da atualização, e atualizar a lista de atividades. Em caso de falha, avisar o usuário sobre esta situação mas permanecer na página atual, sem limpar os dados digitados.

Para atualizar dados no banco de dados, o *site* deve acessar a rota `/` da API com uma requisição do tipo PUT, informando os dados no corpo e na rota da requisição realizada, de acordo com as regras de uma API Restful.

Para obter dados no banco de dados, o *site* deve acessar a rota `/` da API com uma requisição do tipo GET, informando os dados na rota da requisição realizada, de acordo com as regras de uma API Restful.

RF05 - Remoção de atividade

Todas as atividades não realizadas, exibidas na tela inicial, devem possuir uma opção de exclusão, que deve ser confirmada com o usuário. Quando esta opção é confirmada, deve-se remover a atividade escolhida da base de dados, através de uma requisição DELETE para a rota `/` da API, informando os dados na rota da requisição realizada, de acordo com as regras de uma API Restful. Em caso de sucesso, atualizar a lista de atividades e exibir uma mensagem de sucesso ao usuário. Em caso de erro, exibir uma mensagem de erro ao usuário.

RF06 - Visualização de atividades realizadas

Na página principal deve-se exibir um botão (falso, por padrão) em que o usuário indica se deseja visualizar (ou não) as atividades já realizadas. Em caso positivo, as atividades já realizadas devem ser ordenadas por data de realização (que é a data em que ela foi marcada como realizada). Ambas atividades realizadas e não realizadas devem ser exibidas em seções diferentes da página. Em caso negativo, não exibir as atividades já realizadas.

Todas as atividades não realizadas, exibidas na tela principal, devem possuir uma opção para marcar ela como não realizada. Quando esta opção é ativada, essa informação deve ser armazenada no banco de dados, uma mensagem de sucesso/erro deve ser exibida ao usuário, e em caso de sucesso, a atividade deve ser exibida como uma atividade a ser realizada na página inicial.

Para atualizar dados no banco de dados, o *site* deve acessar a rota `/` da API com uma requisição do tipo PUT, informando os dados no corpo e na rota da requisição realizada, de acordo com as regras de uma API Restful.

Para listar os dados obtidos do banco de dados, o *site* deve acessar a rota `/` da API com uma requisição do tipo GET, de acordo com as regras de uma API Restful.

3 Requisitos Não Funcionais

Os requisitos não funcionais descrevem qualidades e características do sistema. Todas os requisitos não funcionais descritos aqui são obrigatórios e devem ser cumpridos no desenvolvimento do sistema.

3.1 RNF01 - Plataforma e uso esperado do sistema

O sistema deve ser acessível por navegadores atuais baseados em HTML5 e Javascript.

3.2 RNF02 - Arquitetura do sistema

O sistema deve utilizar três componentes, sendo um *site* (ver RNF03), uma API (ver RNF04) e um banco de dados (ver RNF05). O *site* deve se comunicar apenas com a API, enquanto o banco de dados deve ser acessado apenas pela API.

3.3 RNF03 - Características do *frontend*

O sistema deve ser construído utilizando o conceito de *Single Page Application* (SPA). O único carregamento de página aparente para o usuário deve acontecer ao acessar o sistema. Todas as trocas de "página" devem ocorrer utilizando rotas com nomes amigáveis e condizentes com a "página" acessada, fazendo a troca de componente ao invés de um novo carregamento do *site*. Deve ser criada uma página de erro comum, para quando o usuário tentar visitar um endereço ou rota inválida. Não será avaliada a aparência do *site*. O *site* deve ser criado com o *framework* Next.js do React na sua versão 14.2.16.

3.4 RNF04 - Características da API

A API para controle de informações deve ser uma API RESTful, escrita em C# utilizando o conceito de API mínima. Os dados de armazenamento devem ser acessados apenas pela API utilizando requisições assíncronas e a biblioteca *Object Relational Mapping* (ORM) Entity Framework. A API não deve permitir acesso direto por código Javascript executado em navegadores.

3.5 RNF05 - Características de armazenamento

O armazenamento do sistema deve ser feito pelo SQL Server, na versão instalada no computador do laboratório de informática.